

United International University

Department of Computer Science and Engineering

Data Structures and Algorithms Lab

Assignment — Stack

Total Marks: 100

Problems: 10

Language: C / C++ / Java / Python

Student Name: _____

ID: _____

Section: _____

Instructions

- Implement each problem as a separate C/C++/Java/Python file.
- Do NOT use built-in Stack, Deque, or similar collection classes for the core logic — implement your own array-backed stack.
- Input format and output format must exactly match what is shown in each problem.
- Clearly write the problem number and your student ID as comments at the top of each file.
- Submissions without proper input/output matching will not receive full credit.
- DO NOT copy from classmates — plagiarism will be checked and penalized heavily.
- DO NOT use ChatGPT or any other AI tool to solve the problems.

Difficulty Guide

● Easy (Problems 1–4)	Foundation — basic push/pop, simulation, reversal
● Medium (Problems 5–8)	Application — real patterns, algorithm thinking required
● Tricky (Problems 9–10)	Design & advanced — read hints carefully, think before coding

● Easy

Problem 1: Undo in a Text Editor [8 marks]

 **Real-World Scenario:** Every text editor has an undo feature. Each character typed is recorded. Pressing Ctrl+Z removes the last typed character. Simulate this using a stack.

Task:

Read a sequence of characters one by one. Regular characters are 'typed' and pushed onto the stack. The special character 'U' means undo — pop the most recent character. After processing all input, print the final text from the stack (bottom to top).

Input Format:

Line 1: N (total operations)
Next N lines: one character each (a letter, digit, or 'U')

Output Format:

The final text as a single string, or "(empty)" if nothing remains

Sample Input / Output:

Input	Expected Output
6 helUlo	hello
4 aUbU	(empty)
5 Hi!U!	Hi!

 **Hint:** If 'U' is pressed when the stack is empty, simply ignore it. To print the result, collect from bottom to top.

● Easy

Problem 2: Validate Bracket Pairs in Code [8 marks]

🔗 **Real-World Scenario:** Every compiler checks that all opening brackets have matching closing brackets. Given a line of code (or just brackets), check whether it is balanced.

Task:

Given a string containing bracket characters (), [], { }, determine if every opening bracket is correctly closed in the right order. Print "Balanced" or "Not Balanced".

Input Format:

Line 1: a string (length ≤ 100 , may contain non-bracket characters – ignore them)

Output Format:

"Balanced" or "Not Balanced"

Sample Input / Output:

Input	Expected Output
{{[]}}	Balanced
int x = (a + [b * c]);	Balanced
([]]	Not Balanced
{[]	Not Balanced

💡 **Hint:** Push every opening bracket. When you see a closing bracket, pop the top and check it matches. If it does not, or the stack is empty, return Not Balanced.

● Easy

Problem 3: Browser Back Navigation [10 marks]

 **Real-World Scenario:** When you click Back in a browser, you return to the page you were on before. Simulate this using a stack — every visited URL is pushed, and Back pops the stack.

Task:

Process a sequence of VISIT and BACK commands. VISIT <url> navigates to a new page. BACK returns to the previous page. Print the current page after each command. If BACK is pressed with no history, print "No previous page".

Input Format:

Line 1: N (number of commands)
Next N lines: VISIT <url> OR BACK

Output Format:

After each command, print: Current: <url> or No previous page

Sample Input / Output:

Input	Expected Output
6 VISIT google.com VISIT gmail.com VISIT docs.google.com BACK BACK BACK	Current: google.com Current: gmail.com Current: docs.google.com Current: gmail.com Current: google.com No previous page
3 VISIT uiu.ac.bd BACK VISIT facebook.com	Current: uiu.ac.bd No previous page Current: facebook.com

 **Hint:** Use one stack for history. VISIT pushes the current page before updating. BACK pops the history stack.

● Easy

Problem 4: Reverse a Food Delivery Route [10 marks]

📌 **Real-World Scenario:** A delivery app recorded pickup stops in order. At the end of the day, the driver must return — visiting stops in reverse. Use a stack to reverse the route.

Task:

Given N stop names, push them all onto a stack and pop them to print the reversed order. Print the original order and the reversed order.

Input Format:

Line 1: N
Next N lines: one stop name each

Output Format:

Original: stop1 stop2 ... stopN
Reversed: stopN ... stop1

Sample Input / Output:

Input	Expected Output
4 Mirpur Dhanmondi Gulshan Banani	Original: Mirpur Dhanmondi Gulshan Banani Reversed: Banani Gulshan Dhanmondi Mirpur
1 Uttara	Original: Uttara Reversed: Uttara
3 Farm Gate New Market Azimpur	Original: Farm Gate New Market Azimpur Reversed: Azimpur New Market Farm Gate

💡 **Hint:** Push all stops, then pop one by one. The pop order is automatically the reverse.

Medium

Problem 5: Evaluate a Calculator Expression (Postfix) [12 marks]

 **Real-World Scenario:** Calculators and programming language interpreters evaluate arithmetic in Reverse Polish Notation (postfix) internally. Build the evaluator.

Task:

Given a postfix expression as space-separated tokens, evaluate it using a stack. Numbers are pushed onto the stack; operators (+, -, *, /) pop two operands, compute, and push the result. Print the final answer. Handle division by zero with an error message.

Input Format:

Line 1: the postfix expression (tokens separated by spaces)

Output Format:

Result: X OR Error: division by zero

Sample Input / Output:

Input	Expected Output
3 4 +	Result: 7
5 1 2 + 4 * + 3 -	Result: 14
6 2 /	Result: 3
4 0 /	Error: division by zero

 **Hint:** When you pop two operands for subtraction or division, the first pop is the right operand and the second pop is the left operand. Order matters!

Medium

Problem 6: Spam Call Detector [12 marks]

 **Real-World Scenario:** A phone app receives calls in sequence. It shows the most recent call at the top (reverse arrival order) and flags numbers that call more than once as potential spam.

Task:

Given N incoming call IDs in arrival order: (1) push them onto a stack, (2) pop and print them — this gives newest-first order, (3) list any call ID that appeared more than once as suspected spam. If no duplicates, print "No suspicious numbers".

Input Format:

Line 1: N
Next N lines: one call ID each (integer)

Output Format:

Newest first: id1 id2 ...
Suspected spam: id (X times) OR No suspicious numbers

Sample Input / Output:

Input	Expected Output
6 101 202 101 303 202 101	Newest first: 101 202 303 101 202 101 Suspected spam:101 (3 times)202 (2 times)
3 555 666 777	Newest first: 777 666 555 No suspicious numbers

 **Hint:** Use a HashMap/dictionary to count frequencies as you push. After printing, check which IDs have count > 1.

Medium

Problem 7: Simplify a File System Path [12 marks]

 **Real-World Scenario:** Operating systems resolve paths that contain `.` (current folder) and `..` (go up one folder). Your task is to simplify such a path, just like a terminal does.

Task:

Given an absolute Unix path, simplify it to its canonical form using a stack. Ignore empty components and single dots (`.`). For `..`, pop the stack (go up), unless the stack is empty. Join the result with `'/'` and prepend `'/'`.

Input Format:

Line 1: an absolute Unix path (starts with `'/'`)

Output Format:

The simplified canonical path

Sample Input / Output:

Input	Expected Output
<code>/home/user/../../docs/./notes</code>	<code>/home/docs/notes</code>
<code>/../etc/passwd</code>	<code>/etc/passwd</code>
<code>/a./b/../../../../c/</code>	<code>/c</code>
<code>/</code>	<code>/</code>

 **Hint:** Split the path by `'/'` — you'll get empty strings for consecutive slashes; skip those. Build your result by joining the stack contents.

● Medium

Problem 8: Next Greater Temperature [14 marks]

🔗 **Real-World Scenario:** A weather app wants to show users: for each day, how many days until a warmer day arrives. This is a classic use of a monotonic stack.

Task:

Given N daily temperatures, for each day find how many days you must wait until a warmer temperature. If no warmer day exists, output 0. Print the result array.

Your solution must use a stack. An $O(n^2)$ brute-force nested loop will not receive full marks.

Input Format:

Line 1: N
Line 2: N space-separated integers (temperatures)

Output Format:

N space-separated integers (days to wait)

Sample Input / Output:

Input	Expected Output
7 73 74 75 71 69 72 76	1 1 4 2 1 1 0
4 30 40 50 60	1 1 1 0
4 60 50 40 30	0 0 0 0

💡 **Hint:** Push indices onto the stack. Pop an index when you find a day warmer than `temperatures[that index]`. The answer for that day is `(current index - popped index)`.

● Tricky

Problem 9: Stack with Instant Minimum [14 marks]

🔗 **Real-World Scenario:** A trading system needs to instantly know the minimum stock price currently held in a portfolio stack — without scanning everything each time.

Task:

Design a stack that supports `push(x)`, `pop()`, `peek()`, and `getMin()` — all in $O(1)$ time. `getMin()` returns the smallest element currently in the stack. The minimum must update correctly when the minimum element is popped.

Demonstrate with at least 8 operations that include pushing non-sorted values, checking `getMin()`, popping the minimum, and verifying the new minimum is correct.

Input Format:

A sequence of operations:
PUSH X, POP, PEEK, GETMIN

Output Format:

For POP, PEEK, GETMIN: print the result
For PUSH: print "Pushed X"

Sample Input / Output:

Input	Expected Output
8 operations: PUSH 5 PUSH 3 PUSH 7 PUSH 1 GETMIN POP GETMIN POP	Pushed 5 Pushed 3 Pushed 7 Pushed 1 Min: 1 Popped: 1 Min: 3 Popped: 7

💡 **Hint:** Use a second auxiliary stack that only stores minimums. Push to it when the new value is $\leq$ current minimum; pop from it when the main stack pops that same value.

⚠️ **Watch Out:** The trap: pushing to the min-stack on every push and popping it on every pop works but wastes space. Can you push to the min-stack only when needed? What condition triggers a push to the min-stack, and when exactly should you pop from it?

● Tricky

Problem 10: Decode a Compressed Message [10 marks]

🔗 **Real-World Scenario:** A network protocol compresses messages: $k[\text{text}]$ means text is repeated k times. Encodings can be nested. Your job is to decode such messages using a stack — no recursion allowed.

Task:

Given an encoded string, decode it iteratively using a stack. Parse digit(s) before '[' as the repeat count (may be multi-digit). When you see ']', pop and repeat the current segment, then append to the string below it.

Input Format:

Line 1: the encoded string (no spaces)

Output Format:

The decoded string

Sample Input / Output:

Input	Expected Output
3[ab]	ababab
3[a2[b]]	abbabbabb
2[cd3[e]]	cdeeecdeee
10[a]	aaaaaaaaaa

💡 **Hint:** Use two stacks: one for accumulated strings, one for repeat counts. On '[': push current string and count, reset both. On ']': pop and reconstruct. Draw the state step-by-step for '3[a2[b]]' before coding.

⚠️ **Watch Out:** Multi-digit numbers like 10[will break code that only reads one character at a time for the count. Accumulate digit characters into a string and convert when you hit '['.